

LES BASES ET CONCEPTS DE LA PROGRAMMATION EN PYTHON 3

1. Conventions d'écriture

```
# Ceci est un commentaire délimité par un # non exécuté par le programme
a=2      # Affectation: Python crée la variable a, lui affecte la valeur 2
          # Typage dynamique: Python lui attribue un type: ici a est un entier (int)
print(a*3) # Fonction et argument: Python calcule le produit a fois 3 et passe cette valeur
           en argument à la fonction print()
>>6
```

```
print(type(a))      # Type de la variable a -> int
>><class 'int'>
```

```
print(a,'a')  # a est un entier (int) 'a' est une chaîne de caractères (str) la virgule est un
séparateur qui devient un espace
# L'éditeur de Python reconnaît ce que vous écrivez et attribue une couleur: noir pour une
variable, vert pour un mot réservé, rouge pour un str et violet pour une opération
arithmétique
>>2 a
```

```
b=a/0.5
print(b,type(b))      # Typage dynamique b est un flottant ( float) soit un réel en maths
>>4.0 <class 'float'>
```

```
c='La conca'+'ténation'  # Le signe + avec des str est une concaténation soit un collage de
deux str
print(c)
>>La concaténation
```

```
print(3E-2,4e+3,2**3)  # Notation scientifique: 3 est la mantisse -2 est l'exposant soit
2*10(-2), 2**3          se lit 2 à la puissance 3
>>0.03 4000.0 8
```

CONCEPTS A RETENIR

Python est un langage interprété avec des règles de syntaxe.

L'interpréteur est un logiciel qui traduit votre script en langage machine après avoir vérifié la faisabilité de votre programme.

L'éditeur de Spyder est un simple éditeur de texte qui se limite à colorer le code au cours de la saisie.

Certains mots sont réservés : print , int, float, type...

Les commentaires doivent être explicites et concis.

Les 3 principales variables sont les entiers, les flottants et les chaînes de caractère comme dans tout autre langage.

Python respecte la casse : m est différent de M.

2. Codage binaire, décimal et hexadécimal

```
m=0b1010 # m est un entier codé en binaire naturel commençant par 0b...
print(m,type(m))
>>10 <class 'int'>
```

```
h=0x10
print(h) # h est un entier codé en hexadécimal commençant par 0x...
>>16
```

```
print(bin(5),bin(-15)) # Conversion décimal -> binaire
>>0b101 -0b1111
```

```
print(hex(20)) # Conversion décimal -> hexadécimal
>>0x14
```

```
print(0b1001+0b0001) # En binaire 01 + 01 = 10 soit 2 en décimal
>>10
```

3. Codage universel des caractères

La table ASCII : La correspondance entre un caractère (chiffre, minuscule, majuscule, caractère de contrôle...) est donnée en décimal, hexadécimal et octal (ancêtre de l'hexadécimal soit la base 8)

http://www.asciitbl.com/site_media/ascii/ascii-table-portrait.jpg

```
print(ord('A'),chr(97)) # la fonction ord() renvoie le code, la fonction chr() renvoie le caractère
>>65 a
```

4. Erreurs communes

```
print(a 'a')
# L'éditeur vous guide par exemple en proposant la paire de parenthèses ou les apostrophes mais n'exécute pas le script
# A l'exécution du script, l'interpréteur vérifie la cohérence de votre script et vous indique
```

CONCEPTS A RETENIR

Les 3 principaux codages sont le binaire naturel, le décimal et l'hexadécimal.

$$m = \mathbf{0b1010} = 1.2^3 + 0.2^2 + 1.2^1 + 0.2^0 = 10$$

Risque de confusion entre le 0 et le O !

$$h = \mathbf{0x10} = 1.16^1 + 0.16^0 = 16 + 0 = 16$$

Par défaut, le code décimal est utilisé.

$$20 = 1.16^1 + 4.16^0 = 16 + 4 = \mathbf{0x14}$$

Addition binaire : $0 + 0 = 0$ $0 + 1 = 1$ et $1 + 1 = 10$

Extrait de la table ASCII :

Dec	Hex	Oct	HTML	Chr	Dec	Hex	Oct	HTML	Chr	Dec	Hex	Oct	HTML	Chr
32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
35	23	043	#	#	67	43	103	C	C	99	63	143	c	c

Une suite d'instruction est appelée script ou programme.

notamment les erreurs de **syntaxe**, de type de variables, de nom non défini...quelques exemples:

```
print(a 'a')
  ^
>>SyntaxError: invalid syntax
-----

print(a):
  ^
>>SyntaxError: invalid syntax
-----

print(2*(4+6)/(6-3)
      ^
>>SyntaxError: incomplete input
-----

print('a'+2)
----> 1 print('a'+2)
>>TypeError: can only concatenate str (not "int") to str
-----

print(2/0)

ZeroDivisionError      Traceback (most recent call last)
----> 1 print(2/0)
>>ZeroDivisionError: division by zero
-----

prnt('print')
----> 1 prnt('print')
>>NameError: name 'prnt' is not defined
-----

print(chr('A'))
----> 1 print(chr('A'))
>>TypeError: 'str' object cannot be interpreted as an integer
-----
```

CONCEPTS A RETENIR

Une fonction est suivie d'arguments délimitée par des parenthèses.

La virgule dans une fonction est un séparateur.

Le : délimite un bloc d'instructions (if..., while..., for i in range())

La classique : la parenthèse est orpheline.

L'éditeur colore les paires de parenthèses.

2 est un entier, '2' est un str

La hantise du développeur : la division par 0

La variable ou la fonction n'est pas reconnue.

La fonction chr() attend comme argument un entier.

x=2

```
print(2x+3)
```

```
print(2x+3)
^
```

```
>>SyntaxError: invalid decimal literal
```

```
if n>2: n=n+3
resultat=false
```

```
----> 1 resultat=false
```

```
>>NameError: name 'false' is not defined
```

5. Variables: entier, flottant, booléen, chaîne de caractères, octet, liste, tuple

a=2	#est un entier (integer:int)
b=3.5	#est un flottant (float)
c="Python3"	#est une chaîne de caractères (string: str)
d=False	#est un booléen (bool)
e= b"FFR"	#est une variable de type octet (bytes)
f=[1,2.3,"dd"]	#est une liste de valeurs modifiables (list)
g=(3,5,9)	#est une liste de valeurs non modifiables (tuple)

6. Affectation de variables

x=3.2E+3	#notation scientifique
y=4+2*x	#expression avec produit prioritaire
a,b=3,5.0	#double affectation
x+=3	#équivalent à x=x+3
z=0xA	#notation hexadécimale
y=0b0110	#notation binaire

7. Opérations arithmétiques

x=11	
y=x/2	#division
y=x//2	#division entière renvoie y=5
r=x%3	#reste de la division entière renvoie r=2
z=x**3	#élévation à la puissance 3
n=abs(-3.5)	#valeur absolue
m=pow(4,3)	#renvoie 64 soit 4 à la puissance 3

CONCEPTS A RETENIR

2 fois x s'écrit 2*x

Python respecte la casse : false différent de False.

Un str peut être délimité par des " : " L'informatique pour tous "

Un booléen est une variable logique qui vaut 0 (False) ou 1 (True).

Une liste est un ensemble de variables hétérogènes délimitées par des []. Une liste est modifiable.

Un tuple est un ensemble de variables hétérogènes délimitées par des (). Un tuple n'est pas modifiable.

La notation scientifique est à privilégier.

Division entière ou division euclidienne :

$11 / 2 = 5$ car $11 = (2 * 5) + 1$ et $11 \% 2 = 1$ car $11 = 2 * 5 + 1$

8. Opérations logiques sur booléens

```
a=False
b=True
print((a and b) or (a and not(b)))
```

9. Conversions de types de variables

```
a=int("15")          #renvoie a=15 (int)
b=int(23.3)          #renvoie b=23
c=float(2)            #renvoie c=2.0
d=str(22)             #renvoie d="22" (str)
e=round(23.3)         #renvoie e=24
f=round(23.32,1)      #renvoie f=23.4
```

10. Opérations sur listes et tuples

```
k=[1,2.0,3,"dd"]     #est une liste hétérogène modifiable
a=k[0]               #renvoie a=1 élément de la liste k d'index 0
fin=k[-1]            #renvoie fin="dd" dernier élément de la liste k
L=len(k)             #renvoie L=4 nombre d'éléments de la liste
p=k[1:3]             #renvoie p=[2.0,3] liste des éléments indexés 1 à 3 exclu!
k.append(5)          #ajout d'un élément à la fin
k.remove(2.0)        #suppression de l'élément de valeur 2.0
q=[2,5,3.2,4.5]
r=q.sort()           #renvoie la liste q triée par ordre croissant
s=q.reverse()        #inverse l'ordre des éléments
```

11. Saisie des entrées et affichage des sorties

```
a=input('Saisir un ou plusieurs caractères:')    #a est un string
b=int(input("Saisir une valeur entière:"))        #b est un entier
print("Votre chaîne de caractères est:",a)
print("La valeur entière saisie est:",b)
print("Terminé\n")
```

```
>>>Votre chaîne de caractères est: dfgwg
>>>La valeur entière saisie est: 12
>>>Terminé
```

CONCEPTS A RETENIR

Logique combinatoire :

and ET logique or OU logique not COMPLEMENT

Table de vérité de ET		
a	b	a ET b
0	0	0
0	1	0
1	0	0
1	1	1

Table de vérité de OU		
a	b	a OU b
0	0	0
0	1	1
1	0	1
1	1	1

Table de vérité de XOR (OU exclusif)		
a	b	a XOR b
0	0	0
0	1	1
1	0	1
1	1	0

Le premier indice d'une liste ou d'un tuple ou d'un str vaut 0. ...1....2

Le dernier indice d'une liste ou d'un tuple ou d'un str vaut -1. -2....-3...

La fonction input() renvoie les caractères saisis au clavier donc des str.

\n indique un saut de ligne

12. Opérations sur chaînes de caractères

```
s="Bla-bla"
L=len(s)          #renvoie le nombre d'éléments de la chaîne de caractères
t=s.upper()        #renvoie en majuscules
t=s.lower()        #renvoie en minuscules
```

13. Définition et appel d'une fonction

```
# fonction qui prend 4 paramètres en entrée et qui renvoie une valeur
```

```
def polynôme(a,b,c,x):
    #Calcule  $y = ax^2 + bx + c$ 
    y=a*x*x + b*x + c
    return y
print(polynôme(1,2,3,2))
```

```
>> 11
```

```
# fonction qui prend 2 paramètres en entrée et qui renvoie 2 valeurs sous la forme d'un tuple
```

```
def calcul(a,b):
    #Calcule  $a+b*b$  et  $a*a + b$ 
    return a+b*b,a*a+b
print(calcul(2,3))
```

```
>>(11, 7)
```

```
# fonction qui ne renvoie pas de valeur
```

```
def affiche(message):
    print(message.upper())
    return None # cette ligne peut être ignorée
affiche("Hello!")
```

```
>>HELLO!
```

CONCEPTS A RETENIR

L'indentation décale l'instruction de 4 espaces ou une tabulation afin de délimiter un bloc d'instruction.

Une fonction est délimitée par le mot-clé **def** suivi du nom, des parenthèses, des arguments, du :, de la suite d'instructions et du **return**.

```
def ma_fonction( arguments ):
    instructions....
    return résultat
```

Une fonction peut prendre plusieurs arguments ou paramètres et renvoyer plusieurs résultats.

Une fonction qui renvoie plusieurs résultats les mettra sous la forme d'un tuple.

14. Boucle itérative conditionnelle : Tant que Condition Faire...

```
masse=12
while masse<100:
    print(masse)
    masse=masse+20
>>12
>>32
...
>>92
```

15. Boucle itérative inconditionnelle : Pour i allant de à

```
for i in range(5):          # Pour i de 0 à 4 faire...
    for j in range( 1,20,2): # Pour j allant de 1 à 20 exclus par pas de 2 faire....
```

16. Tests conditionnels : Si, Si Sinon, Si Sinon Si Sinon...

test avec une condition

```
# Si .....
a=3.2
if(int(a)<4):
    print("Ok") # test vrai donc on affiche Ok
>>Ok
```

```
if(a>2 and a<5):
    print(a) # test vrai donc affiche
>>3.2
```

```
# Si ..... sinon.....
if(a>1):
    print("sup") # test vrai donc affiche sup
else:
    print("inf")
>>sup
```

```
# test avec 2 conditions
# Si ....Sinon si.....Sinon .....
if a>5:
    a=a+2
elif a<2:
    a=a+3
else: a=a+5 # test vrai donc a=8.2
```

CONCEPTS A RETENIR

En pseudo-code, la boucle itérative conditionnelle s'écrit :

Tant que (conditions), faire

En pseudo-code, la boucle itérative inconditionnelle s'écrit :

Pour i de 0 à 4 faire...

Pour j allant de 1 à 20 exclus par pas de 2 faire....

En pseudo-code, les 3 tests conditionnels s'écrivent :

Si (conditions) faire ...

Si (conditions) faire ...

Sinon faire ...

Si (conditions) faire ...

Sinon Si (autres conditions) faire ...

Sinon faire ...

17. Découpage de chaînes de caractères : split()

CONCEPTS A RETENIR

Le découpage d'une chaîne de caractères avec la fonction split() renvoie une liste de str.

```
tram="$GPRMC,071139.988,A,4639.8232,N,00021.6810,E,8.95,184.14,141014,1.1,W,A*65"
champs=tram.split(",")
print(champs)
print("\n\nLatitude brute=",champs[3])
print("Latitude en degrés=",float(champs[3])/100,"°",champs[4])
checksum=champs[-1].split("*")
print("checksum =",int(checksum[1]))
>>['$GPRMC', '071139.988', 'A', '4639.8232', 'N', '00021.6810', 'E', '8.95', '184.14', '141014',
'1.1', 'W', 'A*65']
>>Latitude brute= 4639.8232
>>Latitude en degrés= 46.398232 ° N
>>checksum = 65
```

18. Création de listes : append()

La fonction append() prend un seul argument.

```
tailles=[]
for taille in range(36,48,2):
    tailles.append(taille)
print(tailles)
liste=[]
liste.append('Marc')
liste.append('Sophie')
print(liste)
>>[36, 38, 40, 42, 44, 46]
>>['Marc', 'Sophie']
```

19. Slicing de listes, tableaux ou str

```
ma_liste = ['moi','toi', 6, 3.0]
print(ma_liste[:2],ma_liste[2:],ma_liste[1:2],ma_liste[-3:-1])
mon_str='Informatique pour tous'
print(mon_str[:2],mon_str[2:],mon_str[1:2],mon_str[-3:-1])
mon_tableau = np.array([[1,2,3],[4,5,6],[3,2,7],[1,8,9]])
print(mon_tableau[:2],mon_tableau[2:],mon_tableau[1:2],mon_tableau[-3:-1])
```

[1:3] renvoie la liste des éléments d'index 1 inclus à 3 exclu.

[1:] renvoie la liste des éléments d'index 1 au dernier compris.

[:4] renvoie la liste du premier élément à l'élément d'index 4 exclu.

[-3:-1] renvoie la liste des éléments d'index -3 inclus à -1 exclu.

```
>>['moi', 'toi'] [6, 3.0] ['toi'] ['toi', 6]
>>Informatique pour tous n ou
>>[[1 2 3]
 [4 5 6]] [[3 2 7]
 [1 8 9]] [[4 5 6]] [[4 5 6]
 [3 2 7]]
```


20. Vecteurs et tableaux à 2 dimensions avec numpy

```
import numpy as np
v = np.array([1,2,3,4])          # Vecteur ou tableau à 1 dimension
print(type(v),v[1],np.sum(v),np.size(v),np.shape(v))
t= np.array([[1,2,3],[4,5,6]])
print(type(t),t[1],np.sum(t),np.size(t),np.shape(t))
```

```
>>>class 'numpy.ndarray'> 2 10 4 (4,)
>>>class 'numpy.ndarray'> [4 5 6] 21 6 (2, 3)
```

21. Importation de modules (bibliothèques)

```
import random          # on importe toute la librairie et on écrira random.randint()
from math import sin,pi # on importe les deux modules sin et pi et on écrira directement
                        sin() ou pi()
from random import *    # on importe tous les modules et on écrira directement sin() ou pi()
import matplotlib.pyplot as plt # plt est un alias qui équivaut à matplotlib.pyplot
```

22. Tracé de courbes avec matplotlib

```
# Tracé de y=sin(x) de 0 à 2pi sur 100 points
import matplotlib.pyplot as plt
from numpy import arange,sin,pi
x=arange(0,2*pi,2*pi/100)
y=3*sin(x)
plt.plot(x,y,color='r',linewidth='3')
plt.grid(color='b',linestyle='--',linewidth='1')
plt.title("y = 3.sin(x)")
plt.xlabel("x: angle (degrés) ")
plt.ylabel("y")
plt.show()
```

23. Dictionnaires

```
inventaire={}
inventaire['tablettes']=10
inventaire['écrans']=15
inventaire['portables']=25
print(inventaire)
print("Quantité d'écrans: ", inventaire['écrans'])
del inventaire['tablettes']
print(inventaire)
```

```
# Création d'un dictionnaire vide
# Ajout d'un couple clé-valeur
```

```
# Affichage du dictionnaire
# Affichage de la valeur en donnant la clé
# Suppression d'un élément
```

CONCEPTS A RETENIR

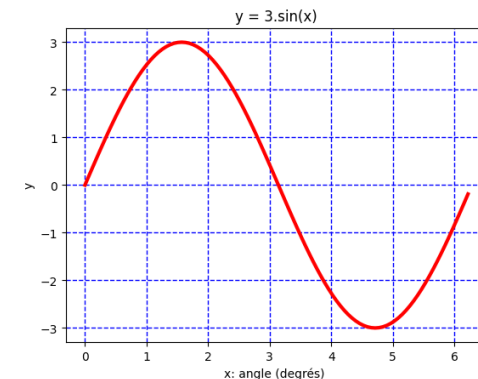
La librairie numpy est spécialisée dans le traitement des vecteurs et tableaux.

La variable est un array.

4 façons d'importer des bibliothèques ou modules

La librairie matplotlib est spécialisée dans le traitement des graphiques.

La fonction arange(début , fin , pas) renvoie une liste de flottants compris entre début et fin par pas.



La variable de type dictionnaire est une liste de couples clé-valeur.

La syntaxe est : {'tablettes': 10, 'écrans': 15, 'portables': 25}

```
inventaire['tablettes']=10
for k in inventaire:
    print("Clé:",k,"Valeur:",inventaire[k])
```

Parcours d'un dictionnaire

```
>>{'tablettes': 10, 'écrans': 15, 'portables': 25}
>>Quantité d'écrans: 15
>>{'écrans': 15, 'portables': 25}
>>Clé: écrans Valeur: 15
>>Clé: portables Valeur: 25
>>Clé: tablettes Valeur: 10
```

24. Algorithmes élémentaires: algorithme d'EUCLIDE pour le calcul du PGCD

Algorithme d'Euclide: calcule le PGCD entre deux entiers

```
def euclide(a:int,b:int):
```

```
    r=a%b
    while r!=0:
        a,b=b,r
        r=a%b
    return b
```

```
x=21
y=15
print("\nLe PGCD de ",x,y," vaut: ",euclide(x,y))
```

```
>>Le PGCD de 21 15 vaut: 3
```

25. Algorithmes élémentaires: La suite de Fibonacci

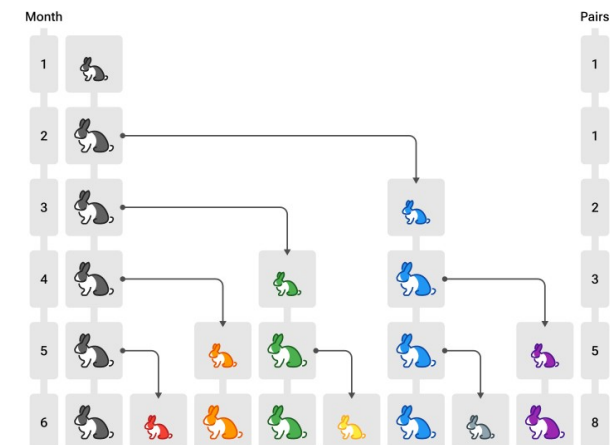
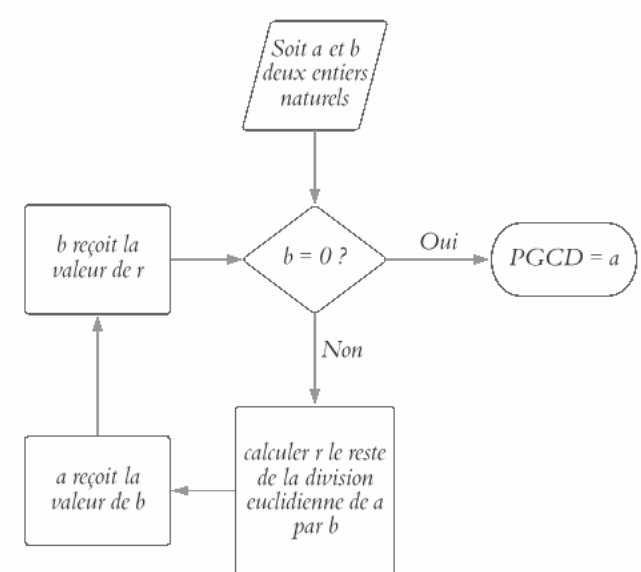
Un terme est la somme des deux précédents

a,b,c = 1,1,1 # affectation parallèle

```
while c<11:
    print(a)
    a,b,c = b, a+b, c+1
```

```
>>1
>>1
>>2
...
>>34
>>55
```

CONCEPTS A RETENIR



26. Algorithmes élémentaires: dérivation numérique méthode d'EULER

Temps intermédiaires en secondes tous les 100 mètres, première valeur temps de réaction, dernière valeur temps total

bolt=[0.183,1.78,1.02,0.9,0.88,0.88,0.85,0.85,0.86,0.86,0.89,9.95]

def vitesse_instantanee2(nom:[]):

Cette fonction renvoie la liste des vitesses instantanées tous les 100m

vit=[0]

for i in range(len(nom)-2):

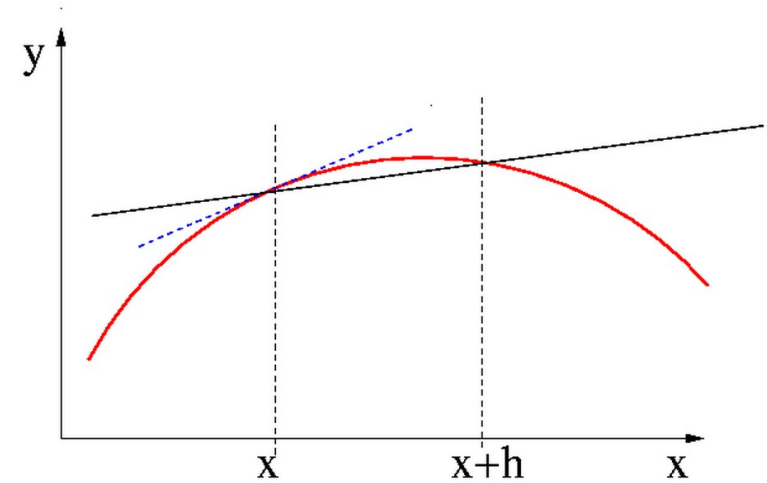
vit.append(round(3.6*10/nom[i+1],2))

return vit

print(vitesse_instantanee2(bolt))

>>[0, 20.22, 35.29, 40.0, 40.91, 40.91, 42.35, 42.35, 41.86, 41.86, 40.45]

CONCEPTS A RETENIR



$$f'(x) = \lim_{(h \rightarrow 0)} \left(\frac{f(x+h) - f(x)}{h} \right)$$

27. Algorithmes élémentaires: intégration numérique par la méthode des RECTANGLES

""Méthode des rectangles,intégrale de f sur intervalle [a,b] pour n points""

def Rectangles(f,a:float,b:float,n:int):

s = 0.0

h = (b-a)/n

for i in range(n):

x = a + i * h

s = s + h*f(x)

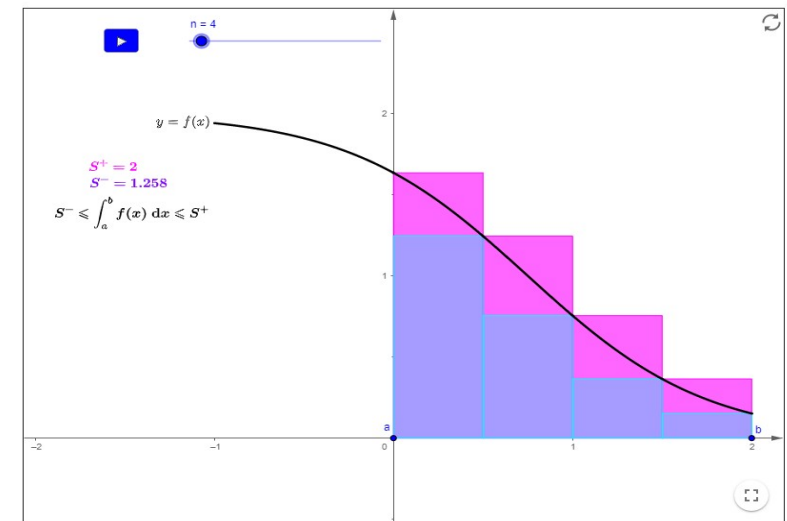
return s

f = lambda x:2*x

result=Rectangles(f,0,2.0,1000)

print("\n\nIntégrale approchée pour 1000 pas",result)

>>Intégrale approchée pour 1000 pas 3.99600000000000013



$$A = \int_a^b f(x) \cdot dx \approx h \cdot \sum_{i=0}^{n-1} f(x_i)$$

28. Algorithmes élémentaires: intégration numérique par la méthode des TRAPEZES

""Méthode des trapèzes,intégrale de f sur intervalle [a,b] pour n points""

```
def trapezes(f,a:float,b:float,n:int):
```

```
    s = 0.0
```

```
    h = (b-a)/n
```

```
    for i in range(n):
```

```
        x = a + i * h
```

```
        s = s + h*((f(x)+f(x+h))/2)
```

```
    return s
```

```
f = lambda x:2*x
```

```
result=trapezes(f,0,2.0,100)
```

```
print("\n\nIntégrale approchée pour 100 pas:",result)
```

>>Intégrale approchée pour 100 pas: 4.0

29. Algorithmes élémentaires: recherche solution $f(x) = 0$

#Recherche solution $f(x)=x^2 - 2 = 0$ avec précision sur x_0

```
a,b,epsilon = 0.0, 2.0, 1E-6
```

```
while (b-a) > epsilon:
```

```
    y0 = (b+a)*(b+a)/4 -2
```

```
    if y0>0:
```

```
        b = (b+a)/2
```

```
    else:
```

```
        a = (b+a)/2
```

```
print(a," < x0 < ",b)
```

>>1.4142131805419922 < x_0 < 1.4142141342163086

#Recherche solution $f(x)=x^2 - 2 = 0$ avec précision sur y_0

```
a,b,epsilon,y0 = 0.0, 2.0, 1E-6,5      # init y0 sinon...
```

```
while abs(y0) > epsilon:
```

```
    y0= (b+a)*(b+a)/4 -2
```

```
    if y0 > 0:
```

```
        b = (b+a)/2
```

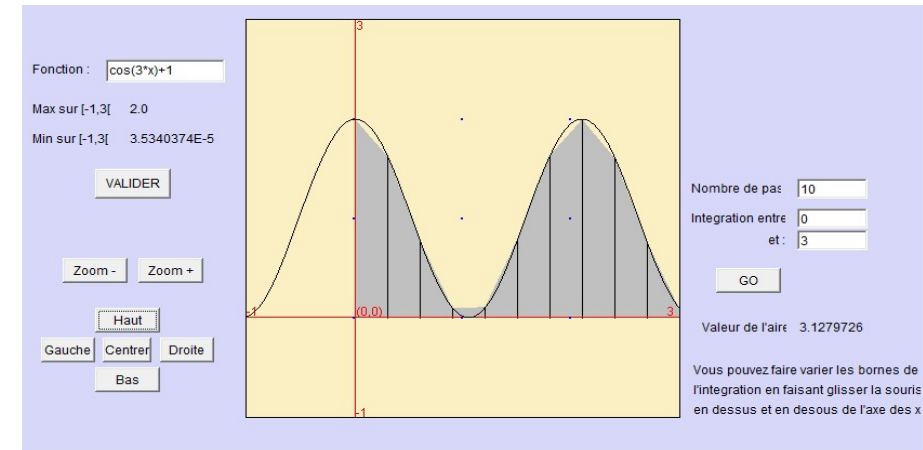
```
    else:
```

```
        a = (b+a)/2
```

```
print(a," < x0 < ",b)
```

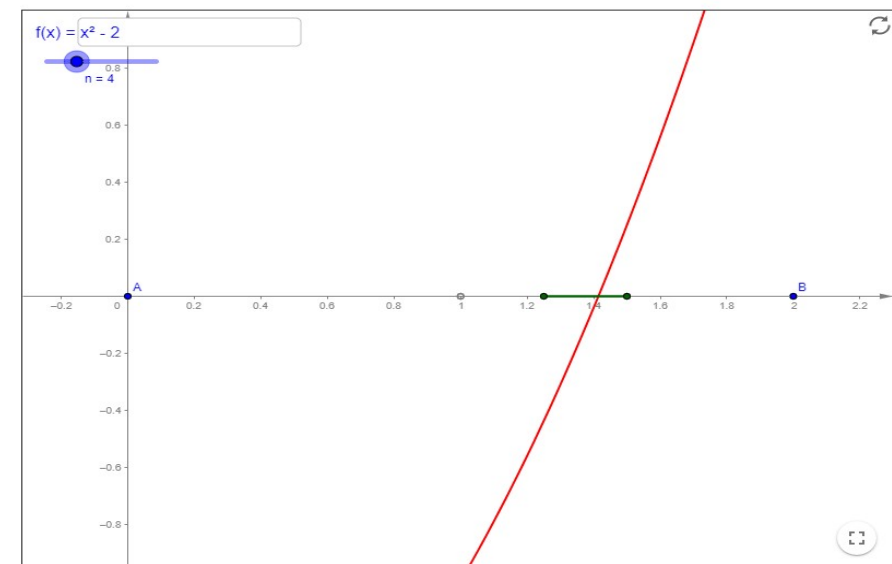
>>1.4142131805419922 < x_0 < 1.4142136573791504

CONCEPTS A RETENIR



$$I = h \sum_{i=1}^n \left(\frac{y_i + y_{(i+1)}}{2} \right)$$

Méthode de la dichotomie : Soit à résoudre l'équation $x^2 - 2 = 0$. La fonction f définie par $f(x) = x^2 - 2$ s'annule sur l'intervalle $[0,2]$ car $f(0)$ et $f(2)$ sont de signes opposés. La recherche de la valeur particulière de x nommée x_0 telle que $f(x_0) = 0$ sera effectuée avec une précision sur $f(x_0)$ de 10^{-6} .



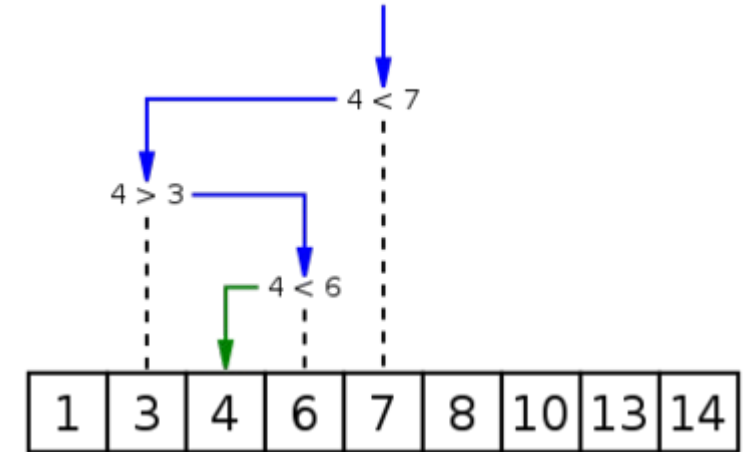
30. Algorithmes élémentaires: méthode de la dichotomie pour la recherche d'une valeur dans une liste triée

CONCEPTS A RETENIR

"Déclaration de la fonction recherche par dichotomie"

```
def dico_liste_triee(x:int,tab:list):  
    # renvoie l'indice de l'élément de la liste tab qui vaut x  
    g,d= 0,len(tab)  
    while g<d-1:  
        m=(d+g)//2  
        print("g=",g,"d=",d,"m=",m)  
        if tab[m]==x:      # si la valeur est trouvée on quitte la boucle while()  
            g=m  
            break  
        if tab[m]>x:  
            d=m  
        else:  
            g=m  
    return g  
print(dico_liste_triee(15,[2,6,8,12,15,25,32,34,36,39,54]))
```

```
>>g= 0 d= 11 m= 5  
>>g= 0 d= 5 m= 2  
>>g= 2 d= 5 m= 3  
>>g= 3 d= 5 m= 4  
>>4
```



31. Ouverture d'un fichier, écriture, lecture et fermeture

```
file=open('materiel.txt','w')  
file.write('Ordinateur portable;10\n')  
file.write('Imprimante couleurs;1\n')  
file.close()
```

```
file=open('materiel.txt','r')  
ligne1=file.readline()  
print(ligne1)  
file.close()
```

```
>>Ordinateur portable;10
```

En programmation orientée objet (POO), file est un objet de type fichier associé à des méthodes comme write() ou close().

Les fichiers de type texte sont éditables avec n'importe quel éditeur de texte dont Spyder et sont ouverts (non propriétaires).

CONCEPTS A RETENIR

```
f=open("prepa.txt","w")  
( write )
```

création du fichier en mode écriture

```
f.writelines(["Matières:\n","Coefficients:\n"])  
print("Ouverture du fichier",f.name)  
f.close()
```

écriture de lignes
fermeture du fichier

Un fichier texte peut être lu en écriture. Il est créé s'il n'existe pas ou en lecture.

```
g=open("prepa.txt","r")  
lecture ( read )  
print(g.readlines())  
g.close()
```

ouverture du fichier existant en mode
lecture des lignes et affichage

```
p=open("prepa.txt","a") # ouverture du fichier en mode ajout ( append )  
p.writelines(["Notes:\n","Colles:\n"]) # écriture de lignes à la suite  
p.close()  
g=open("prepa.txt","r")  
print(g.readlines())  
g.close()
```

lecture des lignes et affichage

```
>>Ouverture du fichier prepa.txt  
>>['Matières:\n', 'Coefficients:\n']  
>>['Matières:\n', 'Coefficients:\n', 'Notes:\n', 'Colles:\n']
```

```
file=open('materiel.txt','r')  
contenu=[]  
for ligne in file:  
    contenu.append(ligne)
```

```
file.close()  
##print(contenu)  
data=contenu[1].split(';')  
print("Type de matériel: "+data[0]+" Quantité: "+data[1])  
import webbrowser  
webbrowser.open('materiel.txt')
```

La librairie webbrowser est spécialisée dans l'ouverture de fichiers quel que soit son format.

```
>>Type de matériel: Imprimante couleurs Quantité: 1
```

32. Création d'une image noir et blanc de format 9 lignes 9 colonnes

```
dam = open('damier.pbm','w')
dam.write("P1#\n")
dam.write("9 9\n")
for i in range(81):
    if i%2==0:
        dam.write("0")
    else:
        dam.write("1")
dam.close()
import webbrowser
webbrowser.open('damier.pbm')
# fichier image à ouvrir avec Gimp
```

33. Création d'une image en niveaux de gris aléatoires 9 par 9

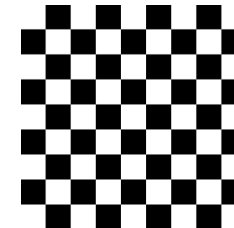
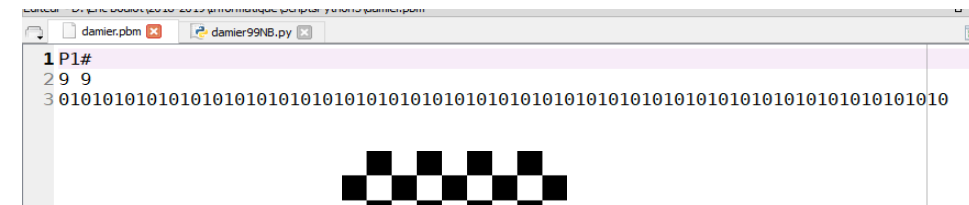
```
from random import randint
alea = open('aleatoire.pgm','w')
alea.write("P2#\n9 9\n")
for i in range(81):
    alea.write(str(randint(1,255))+ "\n")
alea.close()
import webbrowser
webbrowser.open('aleatoire.pgm')
# fichier image à ouvrir avec Gimp
```

34. Création d'une image RGB aléatoire 9 par 9

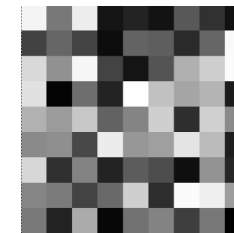
```
from random import randint
rgb = open('image_couleur.pnm','w')
rgb.write("P3#\n9 9\n255\n")
for i in range(81):
    rgb.write(str(randint(1,255))+ "\n")
    rgb.write(str(randint(1,255))+ "\n")
    rgb.write(str(randint(1,255))+ "\n")
rgb.close()
webbrowser.open('image_couleur.pnm')
```

CONCEPTS A RETENIR

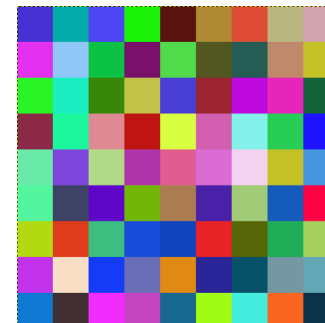
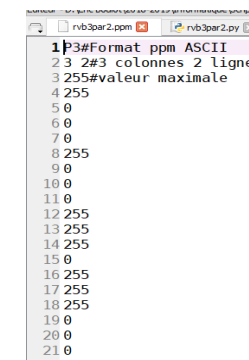
Dans une image Noir et Blanc, un pixel blanc est codé 0, un pixel noir est codé 1.



Dans une image en niveaux de gris, un pixel noir est codé 0, un pixel blanc est codé 255.



Dans une image en couleur, le pixel est codé sur 3 octets RVB.



35. Histogrammes

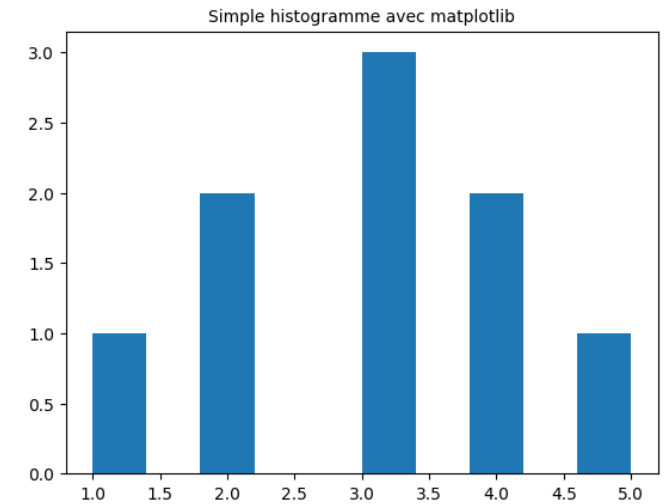
```
# Création, affichage et sauvegarde dans fichier png d'un histogramme
import matplotlib.pyplot as plt
data = [1,2,2,3,3,3,4,4,5]
plt.hist(data)
plt.title('Simple histogramme avec matplotlib', fontsize=10)
plt.savefig("histogramme.png")
plt.show()
```

36. Réponse fréquentielle d'un circuit RC

```
import numpy as np
import matplotlib.pyplot as plt
To=0.001
Te=To/1000
tmax=0.05
t=np.linspace(0,10000,1000)
E=0.1*np.sin(2*np.pi*t)
S=[0.0]
for n in range(999):
    S.append(0.24*E[n+1]+0.761*S[n])
plt.plot(t,E,color='r',linewidth='2')
plt.plot(t,S,color='b',linewidth='2')
plt.grid(color='g',linestyle='--')
plt.xlabel("Temps(us)")
plt.ylabel("En, Sn")
plt.title("Circuit RC réponse fréquentielle")
plt.show
```

CONCEPTS A RETENIR

Un histogramme trace la fréquence (le nombre d'occurrences) pour chacun des items.



La fonction `linspace(min, max, n)` de la librairie `numpy` renvoie une liste de `n` entiers compris entre `min` et `max`.

